Sereact Assignment

Panagiotis Vrachnos

April 12, 2026

1 Introduction

This report presents an end-to-end deep learning pipeline for 3D Bounding Box Prediction using RGB images, organized point clouds, instance segmentation masks, and ground-truth 3D boxes. The goal is to predict the 3D bounding box of each object instance and provide a complete workflow including preprocessing, data loading, training, evaluation, and visualization.

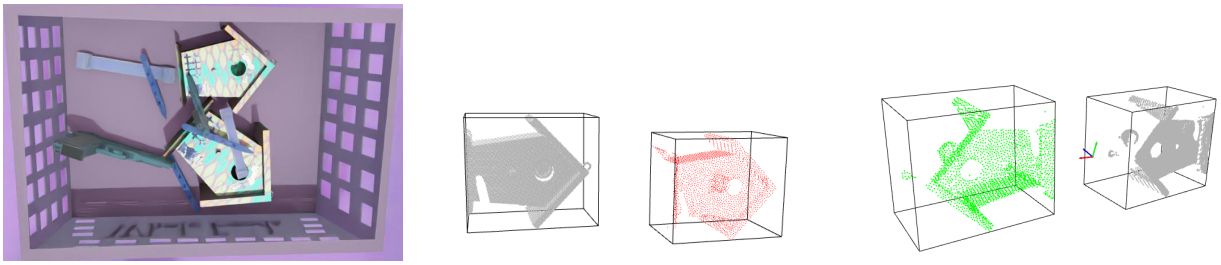
The task is formulated as object-level 3D box regression. Using the provided instance masks, each object is first extracted from the organized point cloud and then fed independently to the model. The network predicts one 3D bounding box per object, represented by its eight corner points. This formulation simplifies the problem by focusing the model on the local 3D geometry of each object instead of the full scene.

2 Data and Preprocess

During preprocessing, each object is extracted independently using its instance mask. The 3D points that belong to the masked pixels are selected from the organized point cloud, and the corresponding RGB values are also retrieved when color is used. Invalid points, such as zero-valued or non-finite coordinates, are removed. This produces one object-level point cloud for each instance in the scene.

Since the number of points varies across objects, each extracted object is converted to a fixed-size representation before training. Objects with many points are downsampled using Farthest Point Sampling, while sparse objects are upsampled by repeating existing points. In this way, all samples share the same input size and can be batched efficiently during training.

A normalization step is then applied to the extracted object points and to the corresponding ground-truth bounding box. This is done in a consistent way so that the model input and the regression target remain in the same coordinate frame. The processed objects, normalized boxes, and meta-data required for later denormalization and visualization are stored in compressed preprocessed scene files.



(a) (b) (c)

Figure 1: Qualitative examples from the preprocessing stage: (a)Original RGB image of a random scene. (b) Extracted object before and after downsampling. (c) Extracted downsampled object before and after normalization

To verify this stage, qualitative visualizations were generated during preprocessing. These include comparisons between the original extracted object and its downsampled version, as well as comparisons between the extracted object and its normalized representation. Additional visualizations of the corresponding ground-truth boxes were also used to confirm that the preprocessing pipeline preserves the overall geometry of the object while producing a model-ready representation.

3 Pipeline Overview

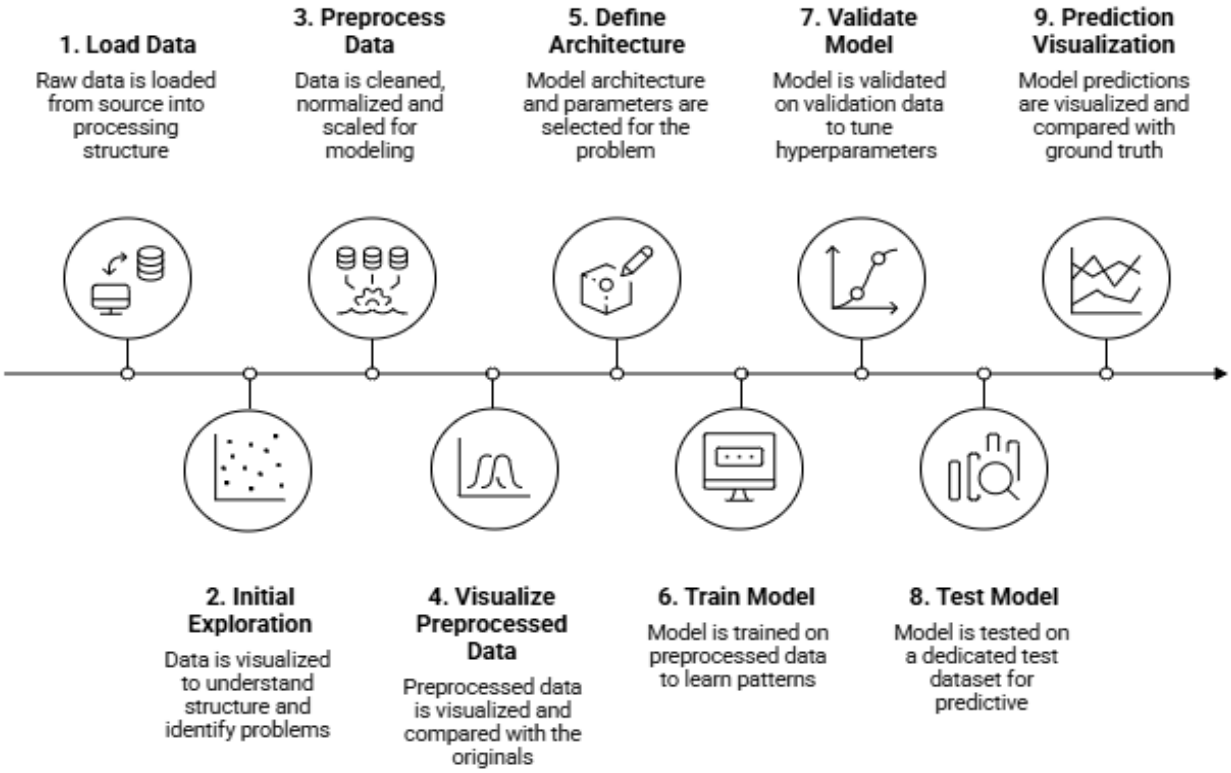


Figure 2: Step-by-step flow of the pipeline

The proposed pipeline follows an object-centered strategy for 3D bounding box prediction. Instead of operating directly on the full scene, the pipeline first transforms the raw scene annotations into object-level samples. This is made possible by the availability of instance segmentation masks, which allow each object to be isolated from the organized point cloud before it is passed to the model.

The workflow begins with preprocessing. For each scene, the instance masks are used to extract the corresponding 3D points from the aligned point cloud. When required, the RGB values of the same pixels are also attached to the extracted points. Invalid or missing points are removed, and each object cloud is converted to a fixed-size representation through downsampling or upsampling. A normalization step is then applied consistently to both the input points and the corresponding ground-truth 3D bounding box. The result of this stage is a set of model-ready object samples stored in preprocessed scene-level files.

These preprocessed samples are then loaded through a custom PyTorch dataset and dataloader. Although the data remains grouped by scene in storage, the dataloader exposes it at object level, so that each training sample corresponds to one segmented object and one target bounding box. This formulation simplifies the learning problem and allows the model to focus on local object geometry rather than performing full-scene multi-object detection.

The network itself is inspired by the PointNet architecture [1]. It receives one object point cloud at a time, extracts point-wise features, aggregates them into a global object descriptor, and predicts the coordinates of the eight corners of the corresponding 3D bounding box. This output representation allows the model to directly regress the final box geometry from the object-centered point cloud.

During training, the predicted box corners are compared with the normalized ground-truth corners through a regression loss. Validation is performed after each epoch in order to monitor convergence and compare different experimental settings. The best-performing configuration is then selected based on validation performance. In the final stage, the chosen checkpoint is evaluated on the test split, and the predicted boxes are transformed back to the original coordinate frame for visualization and qualitative analysis.

4 Experimetal Results

4.1 Model Architecture

In the default geometric setting, each input sample is represented as a fixed-size tensor of 2048×3, where every point is described by its (x, y, z) coordinates. In the experiments that incorporate color information, the input is extended to a tensor of 2048×6, where each point is represented by (x, y, z, r, g, b). In both settings, each object is encoded with a uniform number of points before being passed to the network.

The network consists of a point-wise feature extractor followed by a regression head. The feature extractor is implemented with successive 1×1 convolutions and has the following channel dimensions: 3→64→128→256→512 for the XYZ setting, or 6→64→128→256→512 for the XYZRGB

setting. Each convolution is followed by Batch Normalization and ReLU activation. After the point-wise feature extraction, a global max-pooling layer aggregates the per-point features into a single 512-dimensional global descriptor for the whole object. This descriptor is then passed through a regression head with fully connected layers of dimensions 512→256→128→24, where the final 24 values correspond to the coordinates of the eight predicted box corners, that is, an output of size 8×3. Dropout is applied in the regression head and was also included as an experimental variable.

4.2 Experimental Setup

The model is trained as a direct box regressor. The target is the set of eight ground-truth 3D box corners expressed in the same normalized coordinate frame as the input point cloud. The training loss is Smooth L1 loss computed between predicted and target box corners. This loss was selected because it provides stable optimization for geometric regression while remaining less sensitive to large outliers than a standard L2 loss. The main evaluation metric was the average corner error, defined as the mean Euclidean distance between predicted and ground-truth corners. This metric was used to monitor geometric prediction quality and to compare alternative experimental settings.

Optimization was performed with the AdamW optimizer, using a learning rate of 0.001. The baseline setup used 100 training epochs, dropout of 0.3, and object inputs of size 2048 points. The experiments were conducted on an NVIDIA GeForce RTX 3060 GPU with 12 GB VRAM. Several controlled experiments were carried out in order to study the effect of the main design choices. These included comparisons over the number of epochs, batch size, normalization strategy, dropout, and the use of RGB features in addition to geometry.

The first group of experiments focused on the training schedule and batch size. Increasing the number of epochs from 50 to 100 improved both training convergence and validation performance, indicating that the model benefits from a longer optimization schedule. Batch size also had a strong effect. Among the tested values, as depicted in Figuer3 a batch size of 128 provided the best validation results and achieved a better balance between optimization and generalization than smaller or larger values. Very small batches led to weaker validation performance, while larger batches did not further improve the model and in some cases reduced generalization quality.

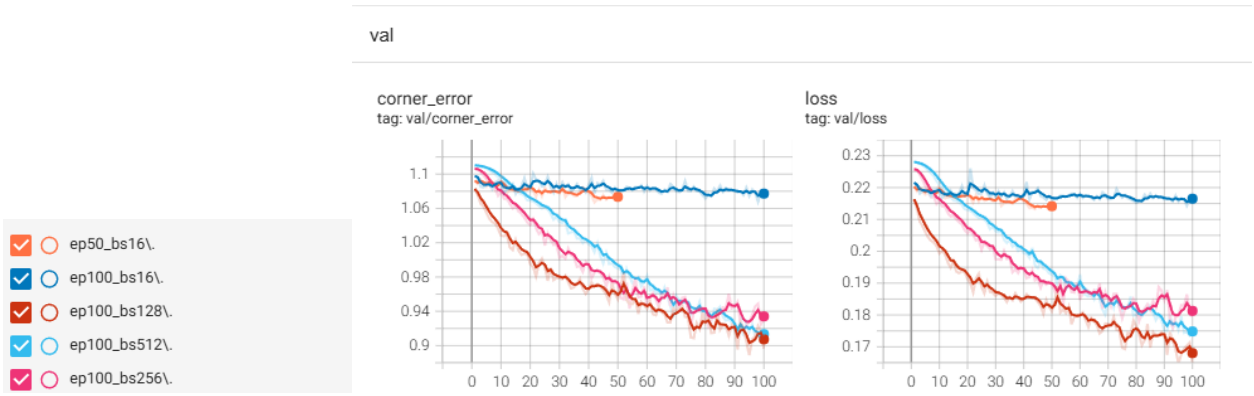


Figure 3: Visualization of corner error and loss during validation. Better performance achieves the experiment with batch size 128

The second group of experiments examined architectural and input-design variations. The influence of dropout was explored by comparing different values in the regression head, while an additional experiment evaluated the use of color by extending the input from XYZ to XYZRGB. In this latter setting, the same preprocessed object files were used, while the model and dataloader consumed all six input channels instead of only the first three. This allowed a direct comparison between purely geometric input and a combined geometric-color representation under otherwise identical conditions.

A further experiment investigated the effect of normalization. In the baseline setting, the extracted object point clouds and their corresponding target boxes were normalized using a unit-sphere strategy. An alternative setting used center-only normalization, in which the object was translated to a local reference frame without scale normalization. This comparison was particularly important because the task requires box size prediction, and per-object scaling may reduce the availability of absolute size cues to the model.

The quantitative results showed that the model learns the regression task stably and that the main performance improvements came from longer training and a more suitable batch size. The strongest baseline configuration among the tested settings, as illustrated in Figure8 was obtained with 100 epochs, batch size 128, unit-sphere normalization, XYZ input, dropout 0.3, and AdamW optimization. However, qualitative visualization revealed that acceptable numerical metrics do not always imply visually satisfactory box predictions. For this reason, the experimental analysis combined both quantitative indicators and qualitative box visualizations in order to assess the actual geometric quality of the final predictions. This was especially important for identifying cases where the average corner error improved, while the predicted boxes still appeared misaligned or geometrically inaccurate when rendered in 3D.

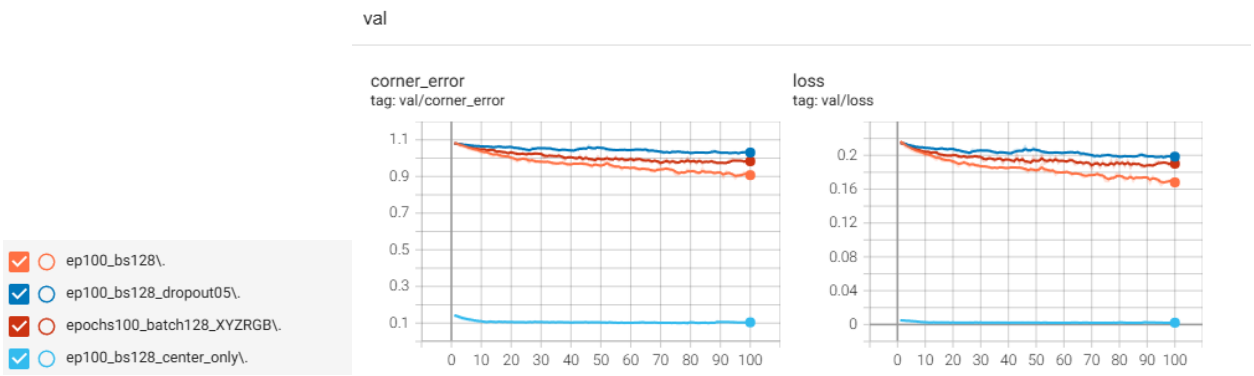


Figure 4: Visualization of corner error and loss during validation for the second group of experiments. Again better performance achieves the experiment with batch size 128

4.3 Qualitative Results

Figure?? illustrate qualitative results of the model on previously unseen test samples. During inference, the predicted bounding boxes are automatically visualized first in the object-centered representation, that is, on the extracted object point clouds used by the model, and then again in the original scene after transformation back to the scene coordinate frame. This two-stage visualization process provides a clearer interpretation of the predictions, supports qualitative assessment of geometric alignment at both local and scene level, and helps identify strengths and failure cases that may not be fully captured by numerical metrics alone.

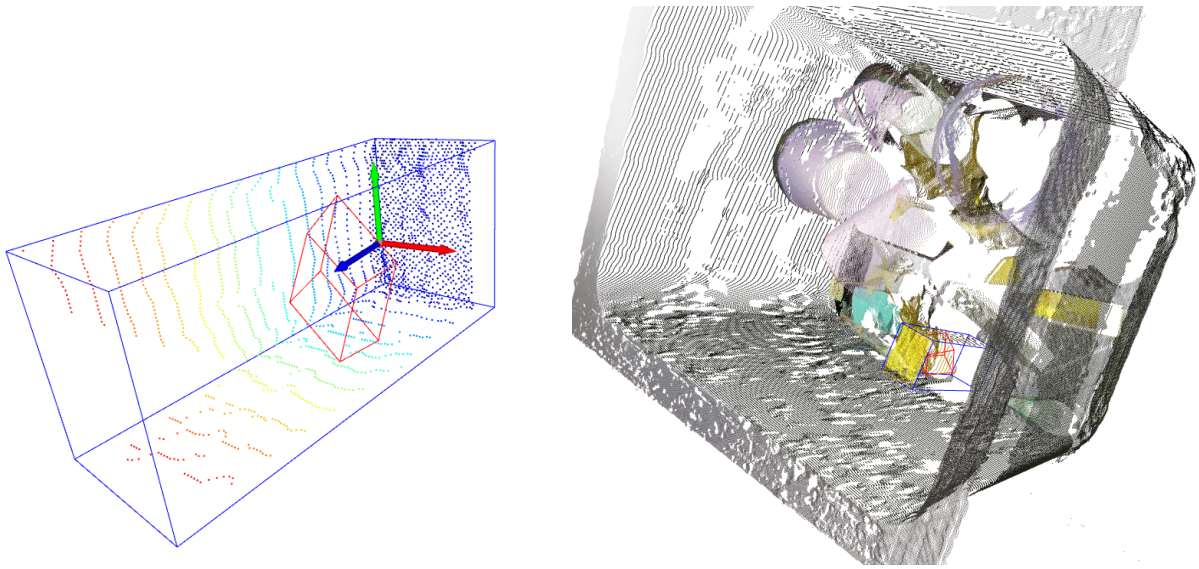


Figure 5

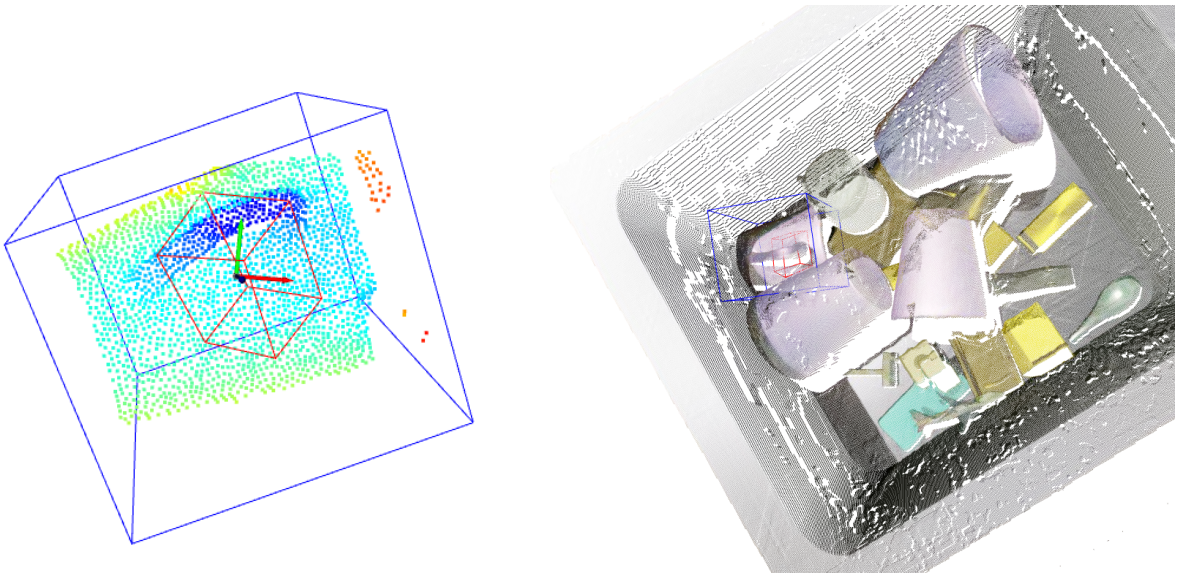


Figure 6

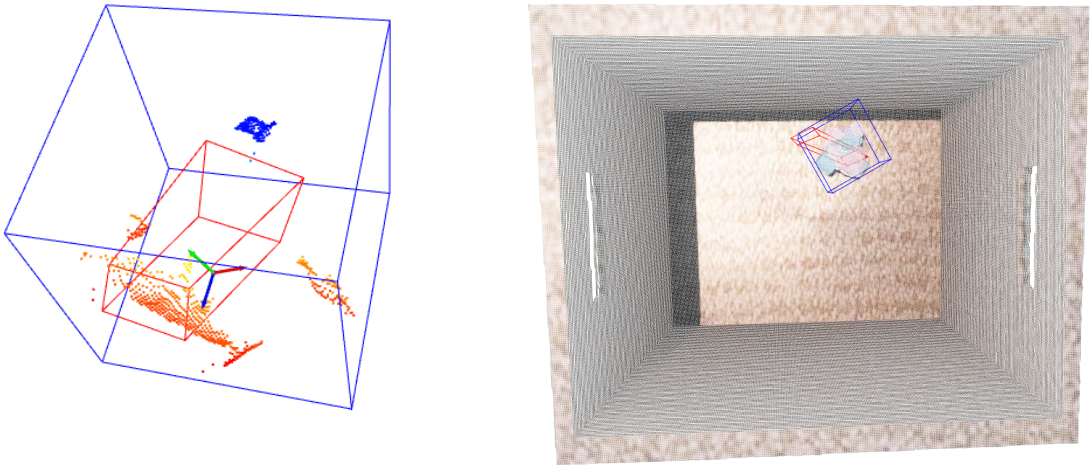


Figure 7

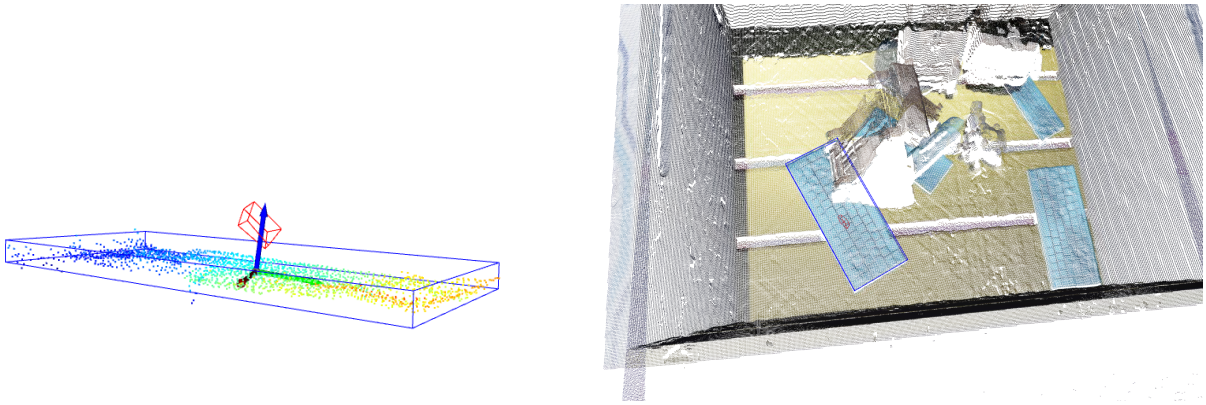


Figure 8

5 Future Work

A possible direction for future work would be to further improve the geometric quality and robustness of the predicted boxes through a richer training strategy. Given the limited time budget of seven days, the focus of this work was on building a complete and functional end-to-end pipeline rather than exhaustively exploring all modeling options. For this reason, several promising extensions remain open for further study. These include the use of data augmentation to improve generalization, the combination of multiple complementary loss terms instead of relying only on corner regression loss, and the integration of IoU-based objectives to better align optimization with the final box overlap quality. More structured box parameterizations and stronger point-cloud architectures could also be investigated in order to produce predictions that are not only numerically competitive but also more geometrically consistent in visual evaluation.

References

[1] Charles R. Qi, Hao Su, Kaichun Mo, and Leonidas J. Guibas. PointNet: Deep learning on point sets for 3D classification and segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 652–660, 2017.